AI Project – Report.

15 – 01 – 2025.

Project made by:

- Meryem Missaoui Hajib
- Mehdi Alkadi
- Ouafae Hmajou
- Amr Agouriane

Professor: Hafidi Hakim

I. Introduction:

The **Smart Expense Tracker Chatbot** is a web-based application designed to streamline personal financial management by leveraging the capabilities of modern technologies, such as Retrieval-Augmented Generation (RAG) and a chatbot interface. The project integrates multiple functionalities, including expense categorization, detailed transaction tracking, CSV data uploads, and a user-friendly conversational interface, to assist users in gaining meaningful insights into their spending habits.

This application aims to provide users with an intuitive and interactive platform where they can:

1. **Track Transactions**: Add, edit, delete, and view transactions stored in a structured database.
2. **Upload CSV Files**: Simplify data management by allowing the upload of financial transactions from CSV files.
3. **Conversational Interface**: Interact with the application via a chatbot to ask questions like "What is my highest expense category?" or "Which month had the most expenses?".
4. **Data Insights**: Use advanced language models to retrieve and generate personalized financial insights using stored transaction data.

The project combines state-of-the-art technologies and best practices in web development and artificial intelligence. By integrating a robust backend and an aesthetic frontend, the application provides a seamless user experience while maintaining the flexibility to handle large amounts of financial data.

This tool serves as a proof-of-concept for how artificial intelligence can assist in personal finance, offering both technical sophistication and practical utility. Whether for individual use or as a foundation for larger financial tracking systems, this project highlights the potential of AI-powered applications in addressing everyday challenges.

II. System Architecture & Design.

a. Overview.

The **Smart Expense Tracker Chatbot** is designed as a modular, web-based application that combines robust backend architecture, an intuitive frontend interface, and advanced AI capabilities for financial management. The system architecture is centered around scalability, user-friendliness, and flexibility, enabling users to seamlessly interact with the platform for managing expenses and gaining financial insights.

The application consists of the following core components:

1. **Frontend**: A user-friendly HTML interface styled with CSS and responsive design principles to allow for smooth interaction with the chatbot, transaction management tools, and CSV upload functionality.
2. **Backend**: Built using Flask, a lightweight and powerful web framework, to handle business logic, data processing, and communication with the database and RAG pipeline.
3. **Database**: An SQLite database to store user transactions, offering simplicity and high compatibility with the Flask framework.
4. **AI-Powered Chatbot**: A Retrieval-Augmented Generation (RAG) pipeline that combines stored transaction data with generative AI to provide insightful and contextually relevant answers to user queries.

b. Why Flask over Google Colab?

Flask was chosen as the primary backend framework for the application instead of relying on **Google Colab** for the following reasons:

1. **Web Application Deployment:**
   - Flask provides a production-ready environment for building scalable web applications, enabling deployment on local servers.
   - Google Colab is primarily designed for prototyping and research rather than hosting full-fledged web applications.
2. **User Experience:**
   - Flask allows for the development of a dedicated and interactive web interface tailored to the needs of the users, such as managing transactions and interacting with the chatbot.
   - Google Colab relies on a notebook-style interface, which is less intuitive and not ideal for end-users unfamiliar with programming environments.
3. **Flexibility and Integration:**
   - Flask integrates seamlessly with frontend tools (HTML, CSS, JavaScript) and databases (like SQLite), providing a robust environment for full-stack development.
   - While Google Colab supports AI model execution, it lacks direct support for creating cohesive web applications with structured backend logic.
4. **Deployment and Accessibility:**
   - Flask applications can be hosted on web servers and accessed by users anywhere through a browser, ensuring better accessibility.
   - Google Colab, on the other hand, requires users to log in and run the notebook themselves, which is not practical for a production-ready application.

5. **Persistent Data Handling:**
    - Flask, in combination with SQLite, ensures persistent data storage for transactions and historical context.
    - Google Colab's runtime is temporary, meaning data is lost when the notebook is closed or the runtime is reset.
6. **Scalability:**
    - Flask applications can scale efficiently, as they are not restricted by the limitations of Colab's runtime environment.

c. Project Architecture.

The **Smart Expense Tracker Chatbot** project is organized into a modular and structured directory system to ensure clarity, maintainability, and scalability. Each folder and file in the project serves a specific purpose, contributing to the functionality of the application. Below is a detailed breakdown of the architecture:

1. backend/

- **Purpose**: This folder contains the core logic of the application, including database management, API routes, and the chatbot logic.
- **Key Files**:
    - **__init__.py**:
        - Ensures that the backend folder is treated as a Python package.
    - **app.py**:
        - The main application logic, handling Flask routes and connecting the frontend with the backend.
        - Manages API endpoints for transactions, CSV uploads, and chatbot interactions.
    - **database.py**:
        - Contains database-related functionality, including CRUD operations (Create, Read, Update, Delete).
        - Defines helper functions for querying and updating the SQLite database.
    - **models.py**:
        - Implements the Retrieval-Augmented Generation (RAG) pipeline for processing user queries.
        - Handles AI-powered chatbot interactions by combining database information with generative AI models.

2. data/

- **Purpose**: Stores persistent data, such as the SQLite database and backups.
- **Key Components**:
    - **expenses.db**:
        - The main SQLite database that holds transaction data, including descriptions, amounts, dates, categories, and transaction types.

3. templates/

- **Purpose**: Contains the HTML templates for the frontend, built using Flask's Jinja2 templating engine.
- **Key Files**:
    - **home.html**:
        - The landing page of the application, providing an overview and navigation links.
    - **transactions.html**:
        - Displays all transactions in a tabular format and allows users to add, edit, or delete transactions.
        - Includes popups for success or error messages during transactions.
    - **upload_csv.html**:
        - Provides an interface for users to upload CSV files containing bulk transaction data.
    - **chat.html**:
        - The chatbot interface where users can interact with the application using natural language queries.
    - **edit_transaction.html**:
        - A form to edit the details of an existing transaction.

4. run.py

- **Purpose**: The entry point for running the application.
- **Functionality**:
    - Initializes the Flask app.
    - Serves the application on a local or production server.
    - Ensures that all necessary components are correctly imported and connected.

Why This Structure?

1. **Separation of Concerns**:

- The backend/ folder isolates the application logic from the presentation layer (templates/).
- Database operations are encapsulated in database.py, while AI logic resides in models.py.

2. **Scalability**:
    - By modularizing the code, the project can easily be expanded with new features or services without affecting existing functionality.
3. **Maintainability**:
    - Each file has a specific purpose, making it easier to debug, test, and update.
4. **Persistence and Safety**:
    - The data/ folder ensures that user data and backups are stored safely, preventing accidental data loss.
5. **User-Friendly Design**:
    - The templates/ folder enables dynamic, responsive, and aesthetic web pages, enhancing user experience.

d. Used Models.

Models Used in the Project

The **Smart Expense Tracker Chatbot** leverages state-of-the-art machine learning models to provide advanced capabilities for understanding and responding to user queries. This section details the two key models integrated into the system: **Google Gemini** for generative AI and **OpenAI Embedding Model** for retrieval-based context generation.

1. Google Gemini

Google Gemini is a generative AI model designed for producing high-quality and contextually accurate text. It plays a crucial role in the chatbot pipeline, enabling the system to generate natural language responses based on user queries and retrieved context.

Role in the Project:

- **Context-Aware Response Generation**:
    - The chatbot uses Gemini to generate responses that incorporate user-provided queries and transaction data retrieved from the database.
    - For example, if a user asks, *"How much did I spend on dining last month?"*, Gemini processes the retrieved data and generates a concise, contextually relevant response.
- **Natural Language Understanding**:

- Gemini is capable of interpreting user intent, ensuring that responses are aligned with the query even if the phrasing is unconventional.

Why Gemini Was Chosen:

- **High-Quality Output**:
  - Gemini is optimized for natural language generation, ensuring that chatbot responses are coherent, accurate, and user-friendly.
- **Context Integration**:
  - It handles large context windows, making it suitable for tasks involving financial data and detailed queries.
- **Ease of Integration**:
  - The Google Generative AI API provides straightforward methods to integrate the Gemini model into Flask applications.

2. OpenAI's text-embedding-3-large

The **text-embedding-3-large** model from OpenAI is a state-of-the-art embedding model that transforms text into numerical vector representations. It forms the backbone of the chatbot's **Retrieval-Augmented Generation (RAG)** pipeline by enabling efficient and accurate data retrieval.

Role in the Project:

- **Vectorization of Text**:
  - Converts transaction data (e.g., descriptions, categories, amounts, dates) into vector representations, making them searchable.
- **Query Embedding**:
  - User queries are transformed into embeddings, which are then compared with the stored transaction embeddings to find the most relevant matches.
- **Efficient Retrieval**:
  - With the help of FAISS (Facebook AI Similarity Search), the system retrieves relevant transactions based on their embedding similarity to the query.

Why text-embedding-3-large Was Chosen:

- **Accuracy**:
  - Produces high-quality embeddings, ensuring accurate matching between queries and transaction data.
- **Scalability**:
  - The model can handle large datasets, making it suitable for applications with extensive financial records.

- **Efficiency**:
    - Embeddings are computationally efficient, enabling fast searches even in large datasets.
- **Proven Performance**:
    - text-embedding-3-large is a well-documented and reliable model for embedding tasks, making it a robust choice for RAG pipelines.

Advantages of Using These Models

1. **Enhanced Query Understanding**:
    - The combination of embeddings and generative AI ensures the chatbot understands both the user's intent and context.
2. **Precision and Relevance**:
    - text-embedding-3-large ensures that only the most relevant transactions are retrieved for response generation.
3. **Human-Like Interaction**:
    - Gemini produces responses that mimic human communication, making the chatbot user-friendly and engaging.
4. **Scalability and Efficiency**:
    - Both models are scalable and efficient, capable of handling extensive transaction datasets and high user traffic.
5. **Seamless Integration**:
    - Both models integrate easily into the Flask application, making the implementation smooth and maintainable.

e. Behind the scenes logic.

The models.py file implements the **Retrieval-Augmented Generation (RAG)** pipeline, which is the core logic behind the chatbot's functionality. This pipeline combines retrieval-based techniques with generative AI to provide accurate and context-aware responses to user queries.

Workflow:

The pipeline operates in **four main steps**, which are modularized into separate functions:

Step 1: Index Corpus from the SQLite Database

Function: index_corpus(corpus)

- **Purpose**:

- Converts transaction data from the database into vector embeddings and indexes them using **FAISS** (Facebook AI Similarity Search) for fast similarity-based retrieval.

- **Process**:

1. **Generate Text Representations**:
    - Each transaction is converted into a structured string, combining fields like description, category, amount, date, and transaction_type.
2. **Generate Embeddings**:
    - The text is passed to OpenAI's **text-embedding-3-large** model to produce vector embeddings that capture the semantic meaning of the text.
3. **Index with FAISS**:
    - FAISS is used to store these embeddings in a searchable format. This enables efficient similarity searches based on user queries.

Step 2: Query and Retrieve Relevant Transactions

Function: retrieve_transactions(query, index, corpus, top_k=None)

- **Purpose**:
    - Matches the user query to relevant transactions in the database by comparing vector embeddings.

- **Process**:

1. **Embed the Query**:
    - The user's query is embedded using OpenAI's **text-embedding-3-large** model, just like the transactions.
2. **Search FAISS Index**:
    - FAISS compares the query embedding to the stored transaction embeddings and returns the top_k most similar matches.
3. **Retrieve Results**:
    - The indices of the matched transactions are used to fetch the corresponding data from the original corpus.

Step 3: Generate Response Using Gemini

Function: generate_response(query, retrieved_context)

- **Purpose**:

- Generates a human-readable response based on the user's query and the retrieved transactions.

- **Process**:

1. **Prepare Context**:
    - The retrieved transactions are concatenated into a single context string.
    - Example:
2. **Generate Prompt**:
    - A prompt is created combining the context and the user's query.
3. **Generate Response**:
    - The prompt is passed to **Google Gemini**'s gemini-1.5-flash model, which synthesizes a detailed and natural language response.

Step 4: Integrate the RAG Process

Function: retrieve_and_generate(query, db_path=db_path, top_k=5)

- **Purpose**:
    - Orchestrates the RAG pipeline by combining the previous steps into a single process.
- **Process**:

1. **Load Corpus**:
    - Fetches transaction data from the database using load_corpus_from_db(db_path).
2. **Index Corpus**:
    - Calls index_corpus(corpus) to prepare embeddings and index the data.
3. **Retrieve Transactions**:
    - Uses retrieve_transactions(query, index, corpus, top_k) to find relevant matches.
4. **Generate Response**:
    - Passes the query and retrieved data to generate_response(query, retrieved_context) to produce the chatbot's response.

f. Sampling Strategy.

In the context of the chatbot, sampling refers to the process of selecting relevant information or responses from a large pool of possible options. For this project, **top-k sampling** is employed during two critical phases of the Retrieval-Augmented Generation (RAG) pipeline:

1. **Retrieving Relevant Transactions** from the database using FAISS.
2. **Generating Natural Language Responses** using Google Gemini.

By using top-k sampling, the system focuses only on the most relevant matches, ensuring efficiency and accuracy in the chatbot's responses.

What is Top-k Sampling?

Top-k sampling is a method where only the top k most probable candidates (or results) are considered for selection. It prioritizes high-probability items, effectively filtering out irrelevant or low-relevance results. This ensures that the system focuses on the most useful data for answering the user's query.

1. Top-k Sampling in Transaction Retrieval

Implementation:

- The function retrieve_transactions() uses top-k sampling when querying the FAISS index.
- FAISS ranks all indexed transaction embeddings based on their similarity to the user's query embedding.
- The **top-k transactions** with the highest similarity scores are returned.

Purpose:

- Limits the results to a manageable and relevant subset of transactions, rather than processing every single transaction in the database.
- Helps the chatbot provide focused and meaningful responses.

This ensures that only the most relevant grocery transactions for the specified time period are returned.

2. Top-k Sampling in Response Generation

Implementation:

- Google Gemini uses **top-k sampling** when generating text responses based on the query and the retrieved transactions.
- The model explores the top k most probable tokens (or words) at each step of response generation, selecting from this narrowed-down pool rather than all possible tokens.

Purpose:

- Ensures the generated response is both relevant and natural-sounding by focusing on high-probability word choices.
- Reduces the likelihood of nonsensical or out-of-context responses.

Why Top-k Sampling is Used

1. **Efficiency**:

- By limiting results or generated tokens to the top k, the system avoids unnecessary computations, making it faster and more resource efficient.

2. **Relevance**:
    - Ensures that only the most meaningful transactions or tokens are considered, improving the accuracy of both retrieval and response generation.
3. **Control**:
    - Top-k sampling introduces a degree of control over randomness in generative models, balancing creativity with relevance.
4. **User Experience**:
    - Prevents the chatbot from being overwhelmed by irrelevant data, enabling it to deliver precise and coherent responses.

How Top-k is Configured

1. **In Transaction Retrieval**:
    - The top_k parameter in retrieve_and_generate() specifies the number of transactions to return.
    - Default value: top_k=5

g. Facebook AI Similarity Check – FAISS.

FAISS (Facebook AI Similarity Search) is a library developed by Meta (formerly Facebook) to perform efficient similarity searches and clustering on large-scale datasets. It is optimized for dense vector similarity and nearest neighbour search, making it a powerful tool for projects involving embedding-based queries like the one implemented in this expense tracker chatbot.

In this project, FAISS is used as the primary indexing mechanism for the **Retrieval-Augmented Generation (RAG)** pipeline to retrieve the most relevant transactions from the database based on user queries.

Why FAISS?

1. **Scalability**:
    - FAISS is designed to handle large-scale datasets efficiently. Even with thousands or millions of embeddings, it performs searches quickly.
2. **Speed**:
    - FAISS uses optimized algorithms, such as clustering and compression techniques, to reduce search time while maintaining high accuracy.
3. **Accuracy**:
    - By leveraging dense embeddings, FAISS ensures the retrieval of highly similar matches, which is crucial for natural language-based queries.

4. **Customizability**:
    - FAISS supports various distance metrics (e.g., cosine similarity, L2 norm), making it adaptable to the needs of different projects.

FAISS in the Project

1. Embedding Transactions

- Each transaction in the database is converted into a dense vector representation using OpenAI's **text-embedding-3-large** model.
- The resulting embeddings encode the semantic meaning of each transaction, including details such as:

2. Indexing with FAISS

- After embedding, FAISS is used to create a **FlatL2 Index**:
    - **FlatL2**:
        - This index stores the embeddings in memory and calculates the L2 norm (Euclidean distance) for similarity search.
        - It is ideal for smaller datasets where exact matches are required.

3. Querying the Index

- When the user inputs a query, it is also converted into an embedding vector using OpenAI's embedding model.
- FAISS searches the index for the top-k most similar embeddings to this query:
    - **Distances**: The similarity scores between the query embedding and transaction embeddings.
    - **Indices**: The indices of the top-k most relevant transactions in the corpus.

FAISS Workflow in the RAG Pipeline

1. **Transaction Embedding**:
    - Each transaction in the database is embedded as a vector using OpenAI's text-embedding-3-large model.
2. **FAISS Indexing**:
    - The embeddings are added to a FAISS index, which stores them for efficient similarity search.
3. **Query Embedding**:
    - User query (e.g., "How much did I spend on groceries in January?") is embedded as a vector.
4. **Similarity Search**:

- FAISS performs a similarity search to identify the most relevant transactions to the query.

5. **Result Retrieval**:
    - The top-k transactions are returned and used to generate the final response.

Advantages of FAISS

1. Real-Time Performance

- FAISS performs similarity searches in milliseconds, making it suitable for real-time applications like chatbots.

2. High Accuracy

- By comparing dense vector embeddings, FAISS ensures that the most semantically relevant transactions are retrieved.

3. Scalability

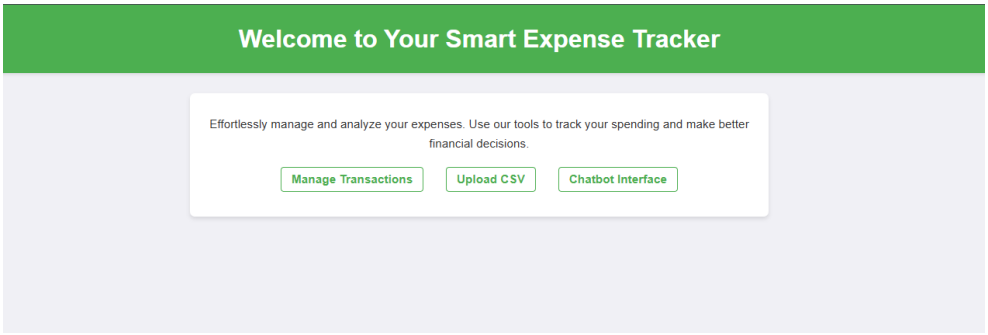
- FAISS can handle large datasets without significant performance degradation. For example:
    - Even with 1 million embeddings, FAISS can perform searches efficiently.

4. Flexibility

- FAISS supports multiple index types:
    - **FlatL2**: Exact search, used in this project for smaller datasets.
    - **IVF (Inverted File Index)**: Approximate search, useful for larger datasets.

III. GUI Overview.

1. Main Menu (Welcome Screen)

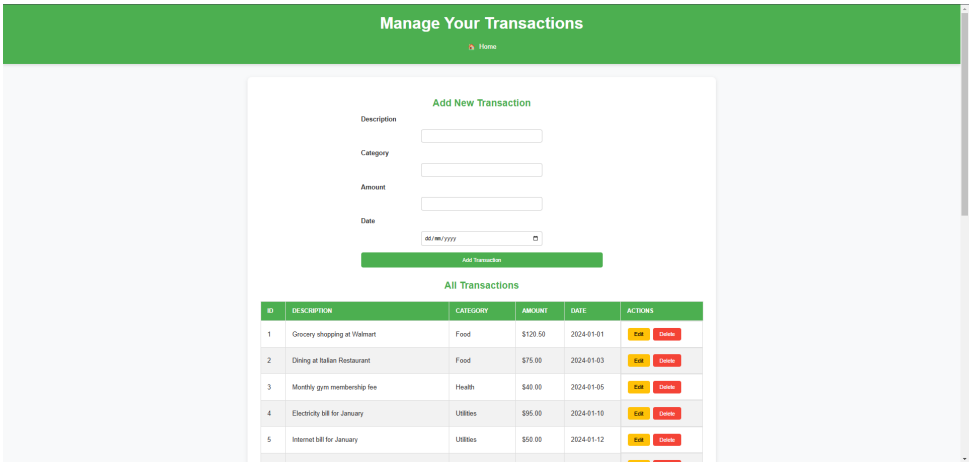


The main menu serves as the central hub of the Smart Expense Tracker application. It provides a simple and intuitive interface for navigating the application's key features. Users are presented with three main options:

1. **Manage Transactions**: Directs the user to the page for viewing, editing, and deleting transactions.

2. **Upload CSV**: Allows users to upload a CSV file containing transaction data for bulk entry into the database.
3. **Chatbot Interface**: Opens the chatbot page, where users can interact with the AI-powered assistant to analyse and inquire about their expenses.

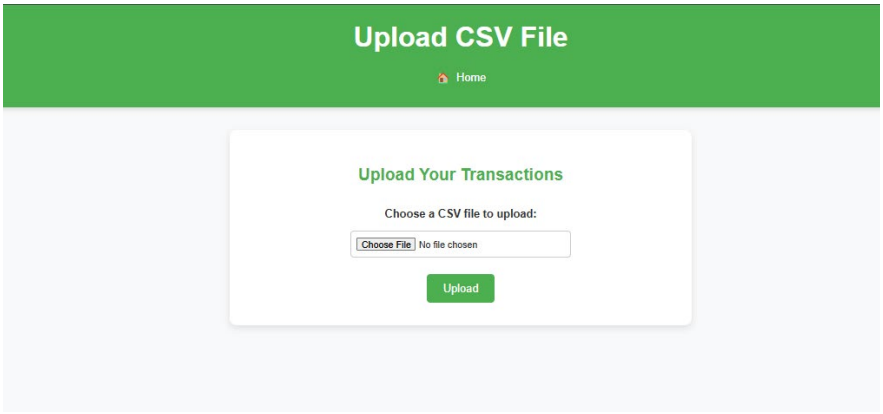
2. Manage Transactions Page.



The Manage Transactions page is designed to provide users with a comprehensive view and control over their financial records. The page includes two key sections:

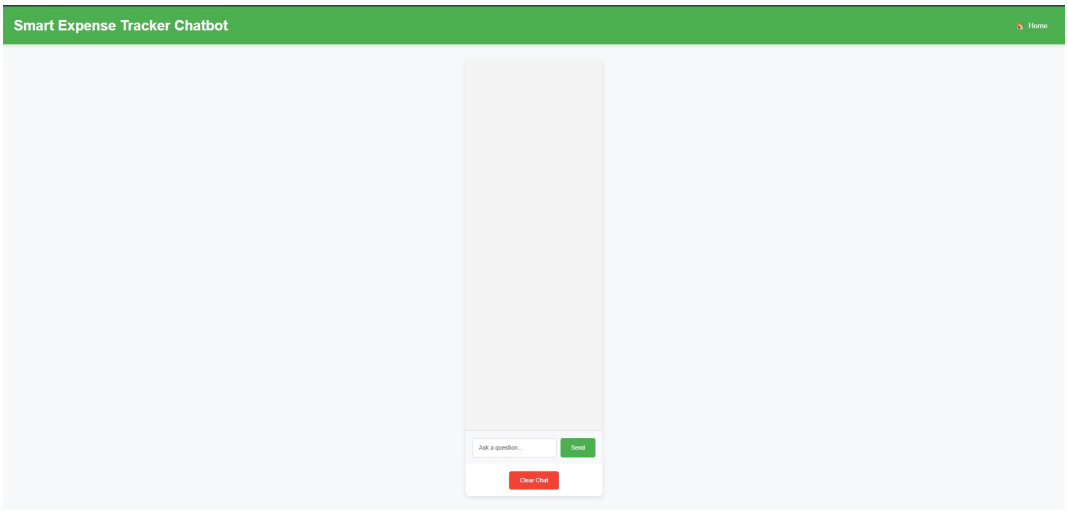
1. **Add New Transaction**:
   At the top, a user-friendly form allows users to quickly add a new transaction by entering:
   - **Description**: A brief explanation of the transaction.
   - **Category**: The type of expense (e.g., Food, Utilities).
   - **Amount**: The monetary value of the transaction.
   - **Date**: The date of the transaction.
     Once submitted, the transaction is added to the list below.
2. **All Transactions**:
   This section displays all existing transactions in a tabular format, with columns for:
   - **ID**: A unique identifier for the transaction.
   - **Description**: A detailed explanation of the expense.
   - **Category**: The type of transaction.
   - **Amount**: The monetary value, displayed in a standardized format.
   - **Date**: The transaction date.
   - **Actions**: Includes "Edit" and "Delete" buttons for easy modification or removal of records.

3. Upload CSV File Page.



The Upload CSV File page allows users to efficiently upload transaction data using a CSV file. The interface is minimalistic and straightforward, consisting of:

1. **Upload Area**:
   A centered panel provides users with:
   - A **file input field** to select a CSV file from their device.
   - An **upload button** to submit the selected file.
2. **Purpose**:
   This page is designed for users who wish to import multiple transactions at once, significantly reducing manual entry time. Upon uploading, the file is processed to extract transaction data and save it to the database. If the upload is successful, a popup notification confirms the operation.
3. **Chatbot Interface**



The chatbot interface is a dedicated space for interacting with the Smart Expense Tracker's intelligent assistant. It provides users with a conversational experience to query and analyze their expenses.

1. **Chat Window**:

- A visually clean, centered, and scrollable chat zone designed to display the user's queries and the chatbot's responses in an easy-to-read format.
- The layout ensures ample space for extended conversations without clutter.

2. **Input Zone**:
    - A **text input box** where users can type their queries, such as:
        - "What was my highest expense last month?"
        - "How much did I spend on groceries in the past year?"
    - A **Send button** to submit the query and receive responses.
3. **Clear Chat Button**:
    - A distinct red button labelled "Clear Chat" allows users to reset the conversation, providing a fresh start for new queries.
4. **Purpose**:
    - The chatbot serves as a key feature for the RAG (Retrieval-Augmented Generation) pipeline, leveraging AI to retrieve relevant transaction data and generate meaningful, context-aware responses.
    - Users can ask complex or detailed questions about their spending habits, and the chatbot will analyse the stored data to provide insights.

IV. Conclusion

The Smart Expense Tracker is an innovative solution designed to simplify and enhance personal financial management through the power of AI and modern web technologies. By integrating a seamless user interface with a robust backend system, the tracker empowers users to effortlessly manage their expenses, gain valuable insights, and make informed financial decisions.

Through the use of Flask for the backend, the project demonstrates flexibility and scalability, ensuring a smooth user experience. The incorporation of advanced AI models, such as OpenAI's text-embedding-3-large for embeddings and Google's Gemini for content generation, provides intelligent, context-aware interactions. The use of FAISS indexing ensures fast and efficient retrieval of relevant data, enabling the chatbot to deliver precise and insightful responses.

From intuitive interfaces for managing transactions to uploading CSV files and interacting with the chatbot, every component is designed with user-centricity in mind. The platform is built to adapt to various user needs, whether it's basic transaction management or querying complex financial patterns.

In conclusion, this project showcases the practical application of cutting-edge AI technologies to address real-world problems. The Smart Expense Tracker not only simplifies financial management but also sets a foundation for future advancements, such as predictive analytics and deeper insights into spending patterns. With its focus on usability, efficiency, and intelligence, this tool is a step forward in making financial decision-making accessible and effortless for all users.